National University of Computer and Emerging Sciences
Islamabad Campus

Parallel and Distributed Computing - (CS3006)

Final Exam

Total Time (Hrs): 2
Total Marks: 105
Total Questions: 8
Date: June 03, 2026

Course Instructors(s):
Dr. Muhammad Arshad Islam, Mr. Fahad Shafique, Mr. Muhammad Aadil Ur Rehman

Section(s): CS (A, B, C, D, E, F, G)

Roll No	Course Section	Student Signature

Important Information: The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.
Do not write below this line.

Attempt all the questions

Part B: Subjective

Instructions:

1. Read the question carefully, understand the question, and then attempt it.
2. All questions must be attempted on the same question paper in the space provided. However, you may ask for an extra sheet for rough work if necessary.
3. After asked to commence the exam, please verify that you have (**19**) different printed pages.
4. No sharing of calculators, notes, or any other material during the exam.
5. Use permanent ink pens only.

	I	II	III	IV	V	VI	VII	VIII	**Total**
CLO #	3	3	3	2	2	2	2	3	
Total Marks	10	10	10	20	10	10	10	25	105
Marks Obtained									

[CLO 3: Perform analytical modelling, dependence, and performance analysis of parallel algorithms and programs.]

Question I . . . . . (10 Marks)

A program is run on 64 processors using Gustafson's Law framework. Measurement shows the serial portion takes 4 seconds and the parallel portion takes 96 seconds (on the 64-processor machine).

(1) **(3 Marks)** Compute the scaled speedup using Gustafson's Law.

Solution: Total time on 64 processors = 4 + 96 = 100s Serial fraction: s = 4/100 = 0.04
a) Gustafson: S = p - s(p - 1) = 64 - 0.04×63 = 64 - 2.52 = 61.48×

(2) **(3 Marks)** What speedup does Amdahl's Law predict for the same p = 64?

Solution: Amdahl: f = 1 - s = 0.96 S = 1 / [0.04 + 0.96/64] = 1 / [0.04 + 0.015] = 1/0.055 ≈ 18.18×

(3) **(4 Marks)** Explain the difference in the two answers.

Solution: Amdahl assumes a fixed workload – with 64 processors the parallel portion per processor is small, so the 4 s serial overhead dominates. Gustafson assumes the workload was chosen to fill all 64 processors, so the serial fraction stays 4% of a much larger total task, giving near-linear speedup.

[CLO 3: Perform analytical modelling, dependence, and performance analysis of parallel algorithms and programs.]

Question II . . . . . (10 Marks)

A startup claims their new parallel framework achieves 90% efficiency at 1000 processors.

(1) **(10 Marks)** Using both Amdahl's and Gustafson's frameworks, analyse whether this claim is plausible

and under what conditions it could be true.

Solution:

Amdahl perspective:
Efficiency E = S/p.
For E = 0.90 at p = 1000, S = 900.
S = 1/[(1-f) + f/1000] = 900 $\Longrightarrow$ (1-f) + 0.001f = 1/900 ≈ 0.00111
(1-f)(1 - 0.001) ≈ 0.00111 - 0.001 $\Longrightarrow$ this gives f ≈ 0.999988, i.e., serial fraction < 0.0012%.
Under Amdahl's fixed-size model this is virtually impossible in practice — any real program has larger serial overhead (I/O, synchronisation). The claim is implausible under strong scaling.

Gustafson perspective:
S(1000) = 1000 - s×999 = 900 $\Longrightarrow$ 999s = 100 $\Longrightarrow$ s ≈ 0.10 (10% serial).
10% serial fraction is realistic for compute-heavy HPC workloads (e.g., molecular dynamics, climate modelling). If the team scales the problem 1000×, the claim is plausible and consistent with Gustafson's model.

Conclusion: The claim requires a weak-scaling interpretation. The startup must clarify that they scale problem size with processors, not that they solve the same problem 900× faster.

[CLO 3: Perform analytical modelling, dependence, and performance analysis of parallel algorithms and programs.]

Question III . . . . . (10 Marks)

Considering the following hypothetical machine and a kernel:

Machine Specifications

System Parameter	Value
Peak compute (π)	512 GFLOP/s
Peak memory bandwidth (β)	64 GB/s
Ridge point (π/β)	8 FLOP/Byte
L2 cache bandwidth	256 GB/s

Kernel Measurements (after running on the machine)

Metric	Measured Value
Floating-point operations	4×10^9 FLOP
Bytes transferred (DRAM)	1×10^9 Bytes
Measured wall-clock time	25 ms

Recalling Roofline Analysis: AI = Total Floating-Point Operations (FLOP) / Total Bytes Transferred (Bytes)
Performance = Total FLOP / Execution Time (seconds)
Performance_ceiling = min(π , AI × β)

If AI < Ridge Point → Memory-bound (performance limited by memory bandwidth)
If AI > Ridge Point → Compute-bound (performance limited by peak FLOP/s)
If AI = Ridge Point → At the balance point (both limits bind equally)

(1) **(2 Marks)** Compute the Arithmetic Intensity (AI) of the kernel.

Solution: AI = FLOP / Bytes = 4×10^9 / 1×10^9 = 4 FLOP/Byte

(2) **(2 Marks)** Compute the measured performance of the kernel in GFLOP/s.

Solution: Performance = FLOP / time = 4×10^9 / 0.025 s = 1.6×10^{11} FLOP/s = 160 GFLOP/s

(3) **(2 Marks)** Determine whether the kernel is compute-bound or memory-bound. Justify your answer by comparing AI to the ridge point.

Solution: Ridge point = 8 FLOP/Byte. AI = 4 < 8 → memory-bound. The kernel cannot exploit peak compute because it does not have enough arithmetic to hide memory latency.

(4) **(2 Marks)** Suppose a code change reduces DRAM traffic by 50% (the FLOP count stays the same). Compute the new Arithmetic Intensity and new performance ceiling.

Solution: New bytes = 0.5×10^9 B. New AI = 4×10^9 / 0.5×10^9 = 8 FLOP/Byte AI = 8 = ridge point → ceiling = min(8×64, 512) = 512 GFLOP/s

(5) **(2 Marks)** Calculate the performance ceiling for this kernel under the roofline model (i.e., the maximum performance it can achieve given its AI).

Solution: Performance ceiling (memory-bound) = AI × β = 4 × 64 = 256 GFLOP/s Efficiency = 160 / 256 = 62.5% (kernel is 37.5% below the memory ceiling)

[CLO 2: Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and OpenMP.]

Question IV . . . . . **(20 Marks)**

Write the output of the following code snippets in the space provided. Consider all required libraries are already imported and the snippet is written inside $main()$ function. In case of multiple answers or an error generalize/explain your answer properly.

(1) **(4 Marks) CODE-1**

```
#define ALIGN 32

int main() {
    // Properly aligned allocation
    float *a = (float*)aligned_alloc(ALIGN, 12 * sizeof(float));
    float *b = (float*)aligned_alloc(ALIGN, 12 * sizeof(float));
    float *c = (float*)aligned_alloc(ALIGN, 12 * sizeof(float));

    // Initialize arrays
    for (int i = 0; i < 12; i++) {
        a[i] = i * 1.0f;
        b[i] = i * 2.0f;
        c[i] = 0.0f;
    }

    // Vectorized computation: process 8 floats per iteration
    __m256 va = _mm256_load_ps(a + 2);   // start at index 2
    __m256 vb = _mm256_load_ps(b + 2);
    __m256 vc = _mm256_fmadd_ps(va, vb, _mm256_set1_ps(1.0f));
    _mm256_store_ps(c + 2, vc);

    // Check result at a specific index
    printf("%f\n", c[5]);
```

Output/Error:

Solution: Mathematically: $(5.0 \times 10.0) + 1.0 =$ **51.0**. But it will most probably crash: Because a float is 4 bytes, shifting the pointer by 2 elements moves the address forward by 8 bytes (2×4).An address that is 32-byte aligned plus 8 bytes is no longer 32-byte aligned. The CPU will trigger a general protection fault when executing the hardware load instruction.

(2) **(4 Marks) CODE-2**

This MPI Program will be executed using 4 processes.

```
    int rank, size, v1, v2, r_target, l_target;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    v1 = (rank + 1) * 10;
    r_target = (rank + 1) % size;
    l_target = (rank - 1 + size) % size;

    if (rank % 2 == 0)
```

```
    {
        MPI_Send(&v1, 1, MPI_INT, r_target, 100, MPI_COMM_WORLD);
        MPI_Recv(&v2, 1, MPI_INT, MPI_ANY_SOURCE, 100, MPI_COMM_WORLD, &status);
    }
    else
    {
        MPI_Recv(&v2, 1, MPI_INT, MPI_ANY_SOURCE, 100, MPI_COMM_WORLD, &status);
        MPI_Send(&v1, 1, MPI_INT, r_target, 100, MPI_COMM_WORLD);
    }

    printf("Rank %d: %d ", rank, v1 + v2);
```

Output/Error:

Solution: Rank 0: 50 Rank 1: 30 Rank 2: 50 Rank 3: 70

(3) **(4 Marks) CODE-3**
This MPI Program will be executed using 4 processes.

```
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_sizes[4] = {1, 3, 2, 2};
    int sendcount = local_sizes[rank];

    int *sendbuf = (int*)malloc(sendcount * sizeof(int));
    for (int i = 0; i < sendcount; i++) {
        sendbuf[i] = ((rank + 2) * (rank + 2)) + (i * 7);
    }

    int recvcounts[4] = {1, 3, 2, 2};
    int displs[4] = {0, 1, 4, 6};

    int total_elements = 8;
    int *recvbuf = NULL;

    if (rank == 0) {
        recvbuf = (int*)malloc(total_elements * sizeof(int));
    }

    MPI_Gatherv(sendbuf, sendcount, MPI_INT,
                recvbuf, recvcounts, displs, MPI_INT,
                0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Process 0 collected: ");
        for (int i = 0; i < total_elements; i++) {
            printf("%d ", recvbuf[i]);
        }
        printf("\n");
```

```
34      }
```

Output/Error:

Solution: Process 0 collected: 4 9 16 23 16 23 25 32

(4) **(4 Marks) CODE-4**
Consider `export OMP_NUM_THREADS=4`

```
1       int main() {
2       int x = 0;
3       #pragma omp parallel
4       {
5           #pragma omp for nowait
6           for (int i = 0; i < 4; i++) {
7               #pragma omp atomic
8               x++;
9           }
10          #pragma omp single
11          x = 100;
12      }
13      printf("%d\n", x);
```

Output/Error:

Solution: Any of the values: 100, 101, 102, or 103.

(5) **(4 Marks) CODE-5**
Consider `export OMP_NUM_THREADS=4`

```
1       int s = r = 0;
2       #pragma omp parallel for reduction(+:s) schedule(static,1)
3       for (int i = 0; i < 4; i++) {
4           s += i;
5       }
6       printf("\ns = %d, r = %d\n", s, r);
```

Output/Error:

Solution: thread0 (i=0) → 0, thread1 (i=1) → 1, thread2 (i=2) → 2, thread3 (i=3) → 3. Sum = 6.

Example possible output: 3 0 2 1 or 0 1 2 3 or 2 0 3 1 etc. s = 6, r = (any value between 0-6))

[CLO 2: Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and OpenMP.]

Question V . . . . . (10 Marks)

You are implementing a real-time audio mixing kernel that combines two input tracks a and b with weights w[0] and w[1], adds a constant scale, and writes to output c. The formula is:

```
c[i] = scale - w[0] * a[i] + w[1] * b[i]
```

The parameters scale, w[0], and w[1] are known at runtime. The arrays are large (**N** = millions of elements). You must write a high-performance AVX2 implementation, considering the following specifications of an available system:

- CPU: Intel Core i7 (Skylake) with AVX2 and FMA Support
- Vector registers: 16 architectural YMM registers (256-bit each)
- SIMD width: 8 single-precision floats per vector (__m256)
- FMA latency: 4 cycles
- FMA throughput: 2 per cycle

(1) **(8 Marks)** Compelete the function `avx_weighted_sum` that utilizes the potential of SIMD capabilities of the system using AVX2 Intrinsics:

- Process the arrays using available AVX2-Vectors.
- Use loop unrolling to improve performance as much as possible.
- Use FMA (_mm256_fmadd_ps) for the arithmetic where possible.
- Use appropriate load/store/broadcast intrinsics.

```
1  void avx_weighted_sum(const float *a, const float *b, float *c,
2                        size_t N, float scale, const float w[2]){
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
```

```
}
```

Solution:

```
#include <immintrin.h>

void avx_weighted_sum(const float *a, const float *b, float *c,
                      size_t N, float scale, const float w[2]) {
    // Broadcast scale and weights to vectors
    __m256 vscale = _mm256_set1_ps(scale);
    __m256 vw0 = _mm256_set1_ps(w[0]);
    __m256 vw1 = _mm256_set1_ps(w[1]);

    size_t i = 0;

    // Choose unroll factor = 4 vectors (32 floats per iteration)
    for (; i + 31 < N; i += 32) {
        // Load 4 vectors from a and b
        __m256 a0 = _mm256_loadu_ps(a + i + 0);
        __m256 a1 = _mm256_loadu_ps(a + i + 8);
        __m256 a2 = _mm256_loadu_ps(a + i + 16);
        __m256 a3 = _mm256_loadu_ps(a + i + 24);

        __m256 b0 = _mm256_loadu_ps(b + i + 0);
        __m256 b1 = _mm256_loadu_ps(b + i + 8);
        __m256 b2 = _mm256_loadu_ps(b + i + 16);
        __m256 b3 = _mm256_loadu_ps(b + i + 24);

        // Compute scale - w0*a + w1*b for each vector
        // Use negative FMA for -w0*a + scale, then positive FMA for + w1*b
        __m256 tmp0 = _mm256_fnmadd_ps(vw0, a0, vscale);
        __m256 c0 = _mm256_fmadd_ps(vw1, b0, tmp0);

        __m256 tmp1 = _mm256_fnmadd_ps(vw0, a1, vscale);
        __m256 c1 = _mm256_fmadd_ps(vw1, b1, tmp1);

        __m256 tmp2 = _mm256_fnmadd_ps(vw0, a2, vscale);
        __m256 c2 = _mm256_fmadd_ps(vw1, b2, tmp2);

        __m256 tmp3 = _mm256_fnmadd_ps(vw0, a3, vscale);
        __m256 c3 = _mm256_fmadd_ps(vw1, b3, tmp3);

        // Store results
        _mm256_storeu_ps(c + i + 0, c0);
        _mm256_storeu_ps(c + i + 8, c1);
        _mm256_storeu_ps(c + i + 16, c2);
        _mm256_storeu_ps(c + i + 24, c3);
    }

    // Scalar tail (handles remaining < 32 elements)
    for (; i < N; i++) {
        c[i] = scale - w[0] * a[i] + w[1] * b[i];
    }
```

```
}
```

(2) **(2 Marks)** How your choice of unrolling factor balances register pressure and latency hiding?

Solution: chose an unroll factor of 4 vectors (32 floats per iteration). The CPU has 16 YMM registers. This unroll uses 4 registers for a, 4 for b, 4 for results, plus 3 for constants – total 15 registers, staying below the limit to avoid spilling. FMA latency is 4 cycles; with 4 independent FMA chains (each chain has two dependent FMAs: first negative then positive), the CPU's out-of-order execution can overlap computation across the 4 chains, effectively hiding the latency. Unrolling by 2 would leave registers idle and not fully hide latency; unrolling by 8 would cause spilling (8×2 = 16 registers for a and b alone, exceeding 16 total).

[CLO 2: Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and OpenMP.]

Question VI . . . . . (10 Marks)

A software engineer is developing a distributed streaming application where P processes are arranged in a logical ring topology. In this system, each process is responsible for filtering out structural data packet bursts and forwarding a varying number of data segments exclusively to its right-hand neighbor (Target), while simultaneously reading a variable-length data segment package from its left-hand neighbor (Source).

To maximize network efficiency and execute this collective data shift in a single call, the engineer decides to use MPI_Alltoallv. The rule for data movement is defined as follows:

- Each Process R sends exactly R + 1 elements from the beginning of its local buffer to its right-hand neighbor.
- Each Process R expects to receive exactly S + 1 elements from its left-hand neighbor (where S is the rank of the source neighbor), placing them at the beginning of its local receive buffer.
- No data is sent to or received from any other processes in the communicator.

(1) **(10 Marks)** Your Task is to Fill in the missing expressions (/* MISSING BLOCK A, B, C, and Output */) in the code below .Your code should be runnable for any number of processes.

```c
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int* sendbuf = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        sendbuf[i] = (rank + 1) * 100 + (i + 1);
    }

    int* sendcounts = (int*)calloc(size, sizeof(int));
    int* sdispls    = (int*)calloc(size, sizeof(int));
    int* recvcounts = (int*)calloc(size, sizeof(int));
    int* rdispls    = (int*)calloc(size, sizeof(int));

    // Step 1: Identify the neighbors in the logical ring topology
```

```
    int target_neighbor, source_neighbor;
    /* MISSING BLOCK A */;

    // Step 2: Configure the non-zero routing constraints for MPI_Alltoallv

    /* MISSING BLOCK B */;

    int* recvbuf = (int*)calloc(size, sizeof(int));

    // Step 3: Execute the irregular communication shuffle

     /* MISSING BLOCK C */;

    // Sequential clean output verification loop
    for (int p = 0; p < size; p++) {
        if (rank == p) {
            printf("Process %d received data from Process %d: ", rank, source_neighbor);
            for (int i = 0; i < recvcounts[source_neighbor]; i++) {
                printf("%d ", recvbuf[i]);
            }
            printf("\n");
        }
        MPI_Barrier(MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Block A:

Block B:

Block C:

Output:

Solution:

```
//Block A:
target_neighbor = (rank + 1) % size;
source_neighbor = (rank - 1 + size) % size;

//Block B:
sendcounts[target_neighbor] = rank + 1;
sdispls[target_neighbor] = 0;

recvcounts[source_neighbor] = source_neighbor + 1;
rdispls[source_neighbor] = 0;

//Block C:
MPI_Alltoallv(sendbuf, sendcounts, sdispls, MPI_INT,
              recvbuf, recvcounts, rdispls, MPI_INT,
              MPI_COMM_WORLD);
Output: (if there are 4 processes)
Process 0 received data from Process 3: 401 402 403 404
Process 1 received data from Process 0: 101
Process 2 received data from Process 1: 201 202
Process 3 received data from Process 2: 301 302 303
```

[CLO 2: Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and OpenMP.]

Question VII . . . . . (10 Marks)

A company is developing a tool to analyze undirected graphs. The graph is represented by an adjacency matrix stored in row□major order, where each entry is either 0 (no edge) or 1 (edge exists). The degree of a node is defined as the edges going inward or outward. The utility must:

- Accept an N×N adjacency matrix (assume N=5 for this example).
- Compute the degree for each node.
- Write the computed degree into an output array.

The host code is provided below. Assume that all required OpenCL headers are included and that the OpenCL platform, device, context, and command queue creation code is provided.

```
    #define N 5

int main() {
    size_t global_work_size = _____________;
    size_t local_work_size = _____________;

    int adjacency[N * N] = {
         0, 1, 0, 1, 0,
         1, 0, 1, 1, 0,
         0, 1, 0, 0, 1,
         1, 1, 0, 0, 1,
         0, 0, 1, 1, 0
    };
    int degrees[N] = {0};

    cl_platform_id platform;
    cl_device_id device;
    cl_context context;
```

```
    cl_command_queue queue;
    cl_program program;
    cl_kernel kernel;
    clGetPlatformIDs(1, &platform, NULL);
    clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL);
    context = clCreateContext(NULL, 1, &device, NULL, NULL, NULL);
    queue = clCreateCommandQueue(context, device, 0, NULL);
    cl_mem bufAdjacency = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
                                         sizeof(int) * N * N, adjacency, NULL);
    cl_mem bufDegrees = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
                                       sizeof(int) * N, NULL, NULL);

    //provide code for compute_degrees.cl file.
    char* kernelSource = readKernelSource("compute_degrees.cl");

    program = clCreateProgramWithSource(context, 1, (const char**)&kernelSource, NULL, NULL);
    clBuildProgram(program, 1, &device, NULL, NULL, NULL);
    kernel = clCreateKernel(program, "compute_degrees", NULL);

    clSetKernelArg(kernel, 0, sizeof(cl_mem), &bufAdjacency);
    clSetKernelArg(kernel, 1, sizeof(cl_mem), &bufDegrees);
    clSetKernelArg(kernel, 2, sizeof(int), &N);


    clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_work_size, &local_work_size, 0, NULL,
    clFinish(queue);
    clEnqueueReadBuffer(queue, bufDegrees, CL_TRUE, 0, sizeof(int) * N, degrees, 0, NULL, NULL);
    for (int i = 0; i < N; i++) {
        printf("Degree of node %d: %d\n", i, degrees[i]);
    }

    // Clean up resources…

    return 0;
}
```

---

(1) **(10 Marks)** Choose **global & local sizes** (ranges) of your choice, mention them in the space provided below. Also, write an optimal kernel function as well.

```
// global_work_size = _____________
// local_work_size = _____________

//compute_degrees.cl Code:
__kernel void compute_degrees(__global const int* adjacency,
                              __global int* degrees,
                              int N)
{
```

```
}
```

Solution:

```
__kernel void compute_degrees(__global const int* adjacency,
                              __global int* degrees,
                              int N)
{
    int i = get_global_id(0);   // node index

    int degree = 0;

    for (int j = 0; j < N; j++) {
        degree += adjacency[i * N + j];
    }

    degrees[i] = degree;
}
```

[CLO 3: Perform analytical modelling, dependence, and performance analysis of parallel algorithms and programs.]

Question VIII ..... (25 Marks)

Perform dependence analysis of the following code and report:

1. Dependence type (true, anti, output, or no dependence)
2. Loop carried/loop independent
3. Distance and Direction vectors
4. Justification

Note: Any answer with "No" or "incorrect" justification will be marked zero even if the other options are correct. In the justification text, you need to show the specific iterations (source and sink) with the relevant data access which causes the dependency (if any). Assumption: all codes are without any error. (This question needs to be solved at the given space below)

(1) **(5 Marks)** .

```
for(int i = 3; i < N-2; i++)
{
    A[i] = B[i+1] + C[i-2];
    D[i-1] = A[i-2] * constant;
}
```

1: [__] True [__] Anti [__] Output [__] No Dependency

2: [__] Loop Independent [__] Loop Carried [__] Not Applicable

3: $d = ()$ $D = ()$ [__] Not Applicable

4: Justification:

Solution: a) Dependence Type: True Dependence (b) Loop Carried/Independent: Loop Carried (c) Distance & Direction Vectors: d = (2), D = (<) (d) Justification: Statement 1 writes A[i]. Statement 2 reads A[i-2] in the same iteration? No, because i-2 is different from i. However, in iteration i+2, statement 1 writes A[i+2] and statement 2 reads A[(i+2)-2] = A[i]. Therefore, the value written at iteration i is read at iteration i+2. This creates a TRUE dependence with distance vector (2) and direction vector (<). The dependence is loop-carried because it spans across multiple iterations.

(2) **(5 Marks)** .

```
for(int i = 2; i < N-1; i++)
    for(int j = 2; j < N-1; j++)
        P[i-2][j+1] = Q[i+1][j-1] - R[i][j] * S[i][j+2];
```

1: [__] True [__] Anti [__] Output [__] No Dependency

2: [__] Loop Independent [__] Loop Carried [__] Not Applicable

3: $d = ()$ $D = ()$ [__] Not Applicable

4: Justification:

Solution: (a) Dependence Type: No Dependence (b) Loop Carried/Independent: Not Applicable (c) Distance Direction Vectors: Not Applicable (d) Justification: The loop writes only to array P. All read arrays are Q, R, and S, which are different from P. No array is both read and written within the loop. Therefore, there are no true, anti, or output dependencies. The loop is completely parallelizable.

(3) **(5 Marks)** .

```
for(int p = 4; p < N-2; p++)
    for(int q = 4; q < N-2; q++)
    {
        A[p-2][q+1] = B[p+1][q-3] + C[p][q];
        B[p-4][q+3] = D[p][q] * constant;
    }
```

1: [__] True [__] Anti [__] Output [__] No Dependency

2: [__] Loop Independent [__] Loop Carried [__] Not Applicable

3: $d = ()$ $D = ()$ [__] Not Applicable

4: Justification:

Solution:

(a) Dependence Type: Anti-Dependence (b) Loop Carried/Independent: Loop Carried (c) Distance Direction Vectors: d = (5, -6), D = (+, -) (d) Justification: Statement 1 reads B[p+1][q-3]. Statement 2 writes B[p-4][q+3] in the same iteration but to different indices (since p+1 ≠ p-4 and q-3 ≠ q+3 for valid p,q). However, the value of B[p+1][q-3] read at iteration (p,q) will be overwritten at iteration (p+5, q-6) because: B[(p+5)-4][(q-6)+3] = B[p+1][q-3] This creates an ANTI-dependence (write-after-read) with distance vector (5, -6) and direction vectors (+, -) meaning p increases and q decreases. The dependence is loop-carried.

(4) **(5 Marks)** .

```
for(int k = 2; k < 15; k++)
{
    A[2*k] = B[k+3] - C[k-1];
    A[3*k - 5] = D[k+1] + E[k];
}
```

1: [__] True [__] Anti [__] Output [__] No Dependency

2: [__] Loop Independent [__] Loop Carried [__] Not Applicable

3: $d = ()$ $D = ()$ [__] Not Applicable

4: Justification:

Solution:

a) Dependence Type: Output Dependence (b) Loop Carried/Independent: Loop Carried (c) Distance Direction Vectors: Not Applicable (non-linear indexing prevents constant distance vector) (d) Justification: Both statements write to the same array A at indices 2k and 3k-5. For output dependence, two different iterations must write to the same A element. Find iterations k_1 and k_2 ($k_1 < k_2$) such that $2k_2 = 3k_1 - 5$: Try $k_1 = 7$: $3(7)-5 = 16$, then $2k_2 = 16 \rightarrow k_2 = 8$ Thus, iteration 7 writes A[16] in statement 2, and iteration 8 writes A[16] in statement 1. Since iteration 7 executes before iteration 8, this creates an OUTPUT dependence (write-after-write). Distance vector is (1) but direction vector is not constant across all iterations due to non-linear indexing, so "Not Applicable" is appropriate.

(5) **(5 Marks)** .

```
  for(int i = 3; i < N-2; i++)
{
    X[i] = Y[i-1] + Z[i+1];
    Y[i+2] = X[i-2] * constant;
    W[i] = Y[i-3] + X[i];
}

for(int j = 4; j < N-3; j++)
{
    X[j+1] = Y[j-2] - W[j+1];
    Z[j] = X[j-1] + constant;
}
```

1: [__] True [__] Anti [__] Output [__] No Dependency

2: [__] Loop Independent [__] Loop Carried [__] Not Applicable

3: $d = ()$ $D = ()$ [__] Not Applicable

4: Justification:

Solution:

a) Dependence Type: True Dependence AND Anti-Dependence (Multiple Dependencies) (b) Loop Carried/Independent: Loop Carried (within first loop AND across both loops) (c) Distance Direction Vectors: For within first loop: d = (2) for X, d = (3) for Y; cross-loop: Not Applicable (d) Justification: Within first loop: • Statement 1 writes X[i]. Statement 2 reads X[i-2] → TRUE dependence from iteration i-2 to i with distance (2), direction (<) • Statement 1 reads Y[i-1]. Statement 2 writes Y[i+2] → ANTI-dependence from iteration i to i+3 with distance (3), direction (<) Across both loops: • First loop writes X[i]. Second loop reads X[j+1] → TRUE dependence when j+1 = i • First loop writes Y[i+2]. Second loop reads Y[j-2] → TRUE dependence when j = i+4 • First loop writes W[i]. Second loop reads W[j+1] → TRUE dependence when j+1 = i • First loop reads Y[i-1]. Second loop writes Y[j-2] → ANTI-dependence when j = i+1

Good Luck!

Cheat Sheet

```c
//OpenMP
int omp_get_thread_num(void);
int omp_get_num_threads(void);
void omp_set_num_threads(int);
double omp_get_wtime(void);
int omp_get_max_threads(void);
void omp_set_nested(int);
void omp_set_dynamic(int);

//MPI
int MPI_Barrier(MPI_Comm comm);
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm);
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm);
int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm);
int MPI_Gatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, const int *recvcounts, const int *displs,
                MPI_Datatype recvtype, int root, MPI_Comm comm);
int MPI_Alltoallv(const void *sendbuf, const int *sendcounts, const int *sdispls,
                  MPI_Datatype sendtype, void *recvbuf, const int *recvcounts,
                  const int *rdispls, MPI_Datatype recvtype, MPI_Comm comm);
int MPI_Comm_set_errhandler(MPI_Comm comm, MPI_Errhandler errhandler);
double MPI_Wtime(void);
int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
```

```
                  int dest, int sendtag, void *recvbuf, int recvcount,
                  MPI_Datatype recvtype, int source, int recvtag,
                  MPI_Comm comm, MPI_Status *status);   // extra
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, int recvcount, MPI_Datatype recvtype,
                int root, MPI_Comm comm);
int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype,
               int root, MPI_Comm comm);
int MPI_Allgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  void *recvbuf, int recvcount, MPI_Datatype recvtype,
                  MPI_Comm comm);
// Handlers: MPI_ERRORS_ARE_FATAL, MPI_ERRORS_RETURN


//OpenCL
cl_mem clCreateBuffer(cl_context context, cl_mem_flags flags,
                      size_t size, void *host_ptr, cl_int *errcode_ret);
cl_int clSetKernelArg(cl_kernel kernel, cl_uint arg_index,
                      size_t arg_size, const void *arg_value);
cl_int clEnqueueNDRangeKernel(cl_command_queue queue, cl_kernel kernel,
                              cl_uint work_dim, const size_t *global_work_offset,
                              const size_t *global_work_size, const size_t *local_work_size,
                              cl_uint num_events_in_wait_list,
                              const cl_event *event_wait_list, cl_event *event);
cl_int clFinish(cl_command_queue queue);
cl_program clCreateProgramWithSource(cl_context context, cl_uint count,
                                     const char **strings, const size_t *lengths,
                                     cl_int *errcode_ret);
cl_int clBuildProgram(cl_program program, cl_uint num_devices,
                      const cl_device_id *device_list, const char *options,
                      void (*pfn_notify)(cl_program program, void *user_data),
                      void *user_data);
cl_kernel clCreateKernel(cl_program program, const char *kernel_name,
                         cl_int *errcode_ret);
cl_int clEnqueueReadBuffer(cl_command_queue queue, cl_mem buffer,
                           cl_bool blocking_read, size_t offset,
                           size_t size, void *ptr,
                           cl_uint num_events_in_wait_list,
                           const cl_event *event_wait_list,
                           cl_event *event);
cl_int clReleaseMemObject(cl_mem memobj);
cl_int clGetKernelWorkGroupInfo(cl_kernel kernel, cl_device_id device,
                                cl_kernel_work_group_info param_name,
                                size_t param_value_size, void *param_value,
                                size_t *param_value_size_ret);

//SIMD Intrinsics

__m256 _mm256_load_ps(float const * mem);
__m256 _mm256_loadu_ps(float const * mem);
void _mm256_store_ps(float * mem, __m256 a);
void _mm256_storeu_ps(float * mem, __m256 a);
__m256 _mm256_set1_ps(float a);
__m256 _mm256_add_ps(__m256 a, __m256 b);
__m256 _mm256_mul_ps(__m256 a, __m256 b);
__m256 _mm256_fmadd_ps(__m256 a, __m256 b, __m256 c);
__m256 _mm256_fnmadd_ps(__m256 a, __m256 b, __m256 c);
```

```
__m256 _mm256_hadd_ps(__m256 a, __m256 b);
__m256 _mm256_sub_ps(__m256 a, __m256 b);
__m256 _mm256_setzero_ps(void);
__m256 _mm256_xor_ps(__m256 a, __m256 b);
__m256 _mm256_cmp_ps(__m256 a, __m256 b, int imm8);
__m256 _mm256_permutevar8x32_ps(__m256 a, __m256i idx);
```